

ЛЕКЦІЯ 8

КОМП'ЮТЕРНА МОДЕЛЬ КЕРУВАННЯ ЗАПАСАМИ

1.МОДЕЛЬ ЕКОНОМІЧНОГО РОЗМІРУ ЗАМОВЛЕННЯ

2.МОДЕЛЬ З ФІКСОВАНОЮ КІЛЬКІСТЮ ЗАМОВЛЕННЯ (Q -СИСТЕМА)

3.МОДЕЛЬ З ФІКСОВАНИМ ІНТЕРВАЛОМ ЗАМОВЛЕННЯ (P -СИСТЕМА)

4. АВС-АНАЛІЗ

5. МОДЕЛЬ ПЕТРІ

6. ВИКОРИСТАННЯ ШТУЧНОГО ІНТЕЛЕКТУ У МОДЕЛЯХ КЕРУВАННЯ ЗАПАСАМИ

Модель керування запасами допомагає визначати, коли і в якій кількості необхідно поповнювати запаси товарів чи ресурсів, щоб мінімізувати витрати на їхнє зберігання та поповнення. Існує кілька основних моделей, які можна використовувати для управління запасами, кожна з яких має свої особливості та підходи до розв'язання задач.

Ось деякі з найпоширеніших моделей:

1. МОДЕЛЬ ЕКОНОМІЧНОГО РОЗМІРУ ЗАМОВЛЕННЯ (Economic Order Quantity –EOQ)

Ця модель використовується для визначення *оптимального розміру замовлення*, що мінімізує сукупні витрати на замовлення та зберігання. Вона підходить для ситуацій, коли *попит є відомим і постійним*.

$$EOQ = \sqrt{\frac{2DS}{H}}$$

де:

D – річний попит на товар;

S – витрати на розміщення одного замовлення;

H – вартість зберігання одиниці товару на рік.

Основні припущення EOQ:

- *Попит є постійним і відомим.*
- *Час виконання замовлення є відомим та фіксованим.*
- *Замовлення прибуває повністю у визначений час.*

Витрати на замовлення і зберігання залишаються стабільними.

Для реалізації моделі керування запасами на Python можна використати модель економічного розміру замовлення (EOQ). Ця програма запитує у користувача річний попит, вартість розміщення одного замовлення та річні витрати на зберігання одиниці товару, а потім обчислює та виводить результат.

[CMM LEC 8-1]

```
import math
```

```

def calculate_eoq(demand, order_cost, holding_cost):
    """
    Розраховує економічний розмір замовлення (EOQ).

    Параметри:
    demand (float): Річний попит на товар (в одиницях).
    order_cost (float): Вартість розміщення одного
замовлення.
    holding_cost (float): Річні витрати на зберігання
одиниці товару.

    Повертає:
    float: Оптимальний обсяг замовлення EOQ.
    """
    return math.sqrt((2 * demand * order_cost) /
holding_cost)

def main():
    # Введення користувачем необхідних даних
    demand = float(input("Введіть річний попит (в
одиницях): "))
    order_cost = float(input("Введіть вартість розміщення
одного замовлення: "))
    holding_cost = float(input("Введіть річні витрати на
зберігання одиниці товару: "))

    # Обчислення EOQ
    eoq = calculate_eoq(demand, order_cost, holding_cost)

    # Виведення результату
    print(f"Оптимальний обсяг замовлення (EOQ): {eoq:.2f}")

```

одиниць ")

```
# Запуск програми  
if __name__ == "__main__":  
    main()
```

Як працює програма:

1. Функція **calculate_eoq** обчислює *EOQ* на основі формули.
 2. Функція **main** запитує вхідні дані, обчислює *EOQ* та виводить результат на екран.
 3. Програма виконується, коли її запускають, і показує оптимальний обсяг замовлення.
-

2. МОДЕЛЬ З ФІКСОВАНОЮ КІЛЬКІСТЮ ЗАМОВЛЕННЯ (*Q*-СИСТЕМА)

Ця модель передбачає, що коли запаси зменшуються до певного рівня (рівня замовлення R), розміщується замовлення фіксованого розміру Q .

Формула для визначення рівня замовлення R :

$$R = d \cdot L + SS$$

де:

d – середній попит за період часу;

L – час виконання замовлення;

SS – страховий запас для захисту від коливання попиту або часу виконання замовлення

Основні припущення Q -системи:

- Розмір замовлення фіксований.
- Замовлення здійснюється тільки тоді, коли рівень запасу досягає певного рівня.
- Підходить для товарів із постійним попитом.

Приведемо приклад програми на Python для реалізації моделі з фіксованою кількістю замовлення (Q -системи). Програма визначає рівень повторного замовлення (R) і обчислює, коли потрібно розміщувати нове замовлення фіксованого обсягу.

Умови завдання:

- Користувач вводить середній попит за одиницю часу, час виконання замовлення та страховий запас.
- Програма перевіряє, чи рівень запасів досяг порогового рівня, і якщо так, видає повідомлення про необхідність нового замовлення.

[СММ 8-2]

```
def calculate_reorder_level(daily_demand, lead_time,
safety_stock):
```

```
    """
```

```
        Розраховує рівень повторного замовлення (Reorder
Level, R) для Q-системи.
```

```
        Параметри:
```

```
        daily_demand (float): Середній попит на товар за
день.
```

```
        lead_time (float): Час виконання замовлення (дні).
```

```
        safety_stock (float): Страховий запас.
```

```
        Повертає:
```

```
        float: Рівень повторного замовлення (R).
```

```
    """
```

```
    reorder_level = daily_demand * lead_time +
safety_stock
    return reorder_level
```

```

def q_system_check(current_stock, reorder_level,
order_quantity):
    """
    Перевіряє, чи потрібно робити нове замовлення, і
    повертає оновлений рівень запасу.

    Параметри:
    current_stock (float): Поточний рівень запасу.
    reorder_level (float): Рівень повторного замовлення.
    order_quantity (float): Фіксована кількість
    замовлення.

    Повертає:
    float: Оновлений рівень запасу.
    """
    if current_stock <= reorder_level:
        print("Рівень запасу досягнув рівня повторного
замовлення. Розміщено нове замовлення.")
        current_stock += order_quantity
    else:
        print("Рівень запасу достатній. Замовлення не
потрібне.")
    return current_stock

def main():
    # Введення параметрів
    daily_demand = float(input("Введіть середній попит на
товар за день: "))
    lead_time = float(input("Введіть час виконання
замовлення (у днях): "))
    safety_stock = float(input("Введіть страховий запас:
"))
    current_stock = float(input("Введіть поточний рівень
запасу: "))
    order_quantity = float(input("Введіть фіксовану
кількість замовлення: "))

    # Розрахунок рівня повторного замовлення
    reorder_level = calculate_reorder_level(daily_demand,
lead_time, safety_stock)
    print(f"Рівень повторного замовлення (R):
{reorder_level:.2f}")

    # Перевірка потреби в замовленні та оновлення запасів
    current_stock = q_system_check(current_stock,

```

```
reorder_level, order_quantity)
    print(f"Оновлений рівень запасу:
{current_stock:.2f}")
```

```
# Запуск програми
if __name__ == "__main__":
    main()
```

Як працює програма:

1. **Функція *calculate_reorder_level*** обчислює рівень повторного замовлення (RRR), використовуючи середній попит, час виконання замовлення і страховий запас.
2. **Функція *q_system_check*** перевіряє, чи поточний рівень запасу досяг рівня повторного замовлення:
 - Якщо рівень запасу низький, розміщується нове замовлення, і кількість товару збільшується на обсяг фіксованого замовлення.
 - Якщо рівень запасу достатній, замовлення не розміщується.
3. **Функція *main*** запитує вхідні дані, обчислює рівень повторного замовлення і виконує перевірку для прийняття рішення.

3. МОДЕЛЬ З ФІКСОВАНИМ ІНТЕРВАЛОМ ЗАМОВЛЕННЯ (P-СИСТЕМА)

У цій моделі замовлення розміщуються через фіксовані інтервали часу, а кількість замовленого товару змінюється залежно від залишку.

Розрахунок обсягу замовлення в моделі з фіксованим інтервалом:

$$Q = d \cdot T + SS - I$$

де:

T – фіксований інтервал між замовленнями;

I – поточний рівень запасів.

Основні припущення P -системи:

- *Замовлення розміщуються через фіксовані проміжки часу.*
- *Обсяг замовлення змінюється залежно від потреби.*

Приведемо приклад програми на Python для реалізації моделі з фіксованим інтервалом замовлення (P -системи), де замовлення розміщуються через певні проміжки часу, і його обсяг залежить від поточного рівня запасів.

Умови завдання:

- *Користувач вводить середній попит за одиницю часу, інтервал між замовленнями, час виконання замовлення та страховий запас.*
- Програма перевіряє поточний рівень запасів і розраховує кількість, яку потрібно замовити для поповнення до потрібного рівня запасу.

[СММ ЛЕС 8-3]

```
def calculate_target_inventory(daily_demand,
review_period, lead_time, safety_stock):
    """
    Розраховує цільовий рівень запасу (Target Inventory
    Level,  $T$ ) для  $P$ -системи.

    Параметри:
    daily_demand (float): Середній попит на товар за
    день.
    review_period (float): Інтервал між замовленнями (у
    днях).
    lead_time (float): Час виконання замовлення (у днях).
    safety_stock (float): Страховий запас.

    Повертає:
    float: Цільовий рівень запасу ( $T$ ).
    """
    target_inventory = daily_demand * (review_period +
lead_time) + safety_stock
    return target_inventory
```



```

def p_system_order_amount(current_stock,
target_inventory):
    """
        Розраховує обсяг замовлення для поповнення запасів до
        цільового рівня.

        Параметри:
        current_stock (float): Поточний рівень запасу.
        target_inventory (float): Цільовий рівень запасу.

        Повертає:
        float: Обсяг замовлення.
    """
    order_quantity = max(0, target_inventory -
current_stock)
    return order_quantity

def main():
    # Введення параметрів
    daily_demand = float(input("Введіть середній попит на
товар за день: "))
    review_period = float(input("Введіть інтервал між
замовленнями (у днях): "))
    lead_time = float(input("Введіть час виконання
замовлення (у днях): "))
    safety_stock = float(input("Введіть страховий запас:
"))
    current_stock = float(input("Введіть поточний рівень
запасу: "))

    # Розрахунок цільового рівня запасу
    target_inventory =
calculate_target_inventory(daily_demand, review_period,
lead_time, safety_stock)
    print(f"Цільовий рівень запасу (T):
{target_inventory:.2f}")

    # Розрахунок обсягу замовлення
    order_quantity = p_system_order_amount(current_stock,
target_inventory)
    print(f"Обсяг замовлення для поповнення запасу:
{order_quantity:.2f}")

# Запуск програми

```

```
if __name__ == "__main__":  
    main()
```

Як працює програма:

1. Функція **calculate_target_inventory** обчислює цільовий рівень запасу (T), використовуючи середній попит, інтервал між замовленнями, час виконання замовлення і страховий запас.
2. Функція **p_system_order_amount** визначає кількість, яку потрібно замовити:
 - Вона обчислює різницю між цільовим рівнем запасу і поточним рівнем запасу.
 - Якщо поточний запас нижчий за цільовий рівень, програма розміщує замовлення, щоб досягти цього рівня.
3. Функція **main** запитує вхідні дані, обчислює цільовий рівень запасу, а потім визначає обсяг замовлення для досягнення цього рівня.

4. ABC-АНАЛІЗ

Цей метод розбиває товари на три групи: **A**, **B** і **C**. Категорія **A** включає товари з високою вартістю, але низьким обсягом, **B** – товари середнього значення, і **C** – товари низької вартості, але великого обсягу. Такий аналіз допомагає зосередити ресурси на управлінні запасами для найважливіших товарів (група **A**), приділяючи менше уваги групам **B** і **C**.

Модель ABC використовується для класифікації запасів на основі їх вартості та частоти використання. Зазвичай, у системі ABC запаси поділяються на три категорії:

- **Категорія A:** запаси з найвищою вартістю, але *зазвичай* низькою частотою використання.
- **Категорія B:** середньої вартості та середньої частоти використання.
- **Категорія C:** товари з низькою вартістю, але високою частотою використання.

Приклад програми Python для ABC-класифікації запасів

Програма, яка приймає список запасів з їх вартістю та кількістю використання, потім розраховує загальну вартість кожного запасу та класифікує запаси за ABC методом.

[CMM LEC 8-4]

```
# Дані про запаси (назва, ціна за одиницю,
кількість)
items = [
    {"name": "Item1", "unit_price": 100,
"quantity": 20},
    {"name": "Item2", "unit_price": 50,
"quantity": 100},
    {"name": "Item3", "unit_price": 10,
"quantity": 300},
    {"name": "Item4", "unit_price": 5, "quantity":
1000},
    {"name": "Item5", "unit_price": 500,
"quantity": 5},
]

# Крок 1: Розрахунок загальної вартості кожного
запасу
for item in items:
    item["total_value"] = item["unit_price"] *
item["quantity"]

# Крок 2: Сорткування за загальною вартістю (від
більшої до меншої)
items.sort(key=lambda x: x["total_value"],
reverse=True)

# Крок 3: Класифікація за ABC методом
total_inventory_value = sum(item["total_value"]
for item in items)
a_threshold = 0.7 * total_inventory_value #
Категорія А: 70% загальної вартості
b_threshold = 0.9 * total_inventory_value #
Категорія В: до 90%
```

```

a_value, b_value = 0, 0
for item in items:
    if a_value < a_threshold:
        item["category"] = "A"
        a_value += item["total_value"]
    elif b_value < b_threshold:
        item["category"] = "B"
        b_value += item["total_value"]
    else:
        item["category"] = "C"

# Виведення результатів
print("Результати ABC-класифікації:")
for item in items:
    print(f"{item['name']}: Category {item['category']}, Total Value: {item['total_value']}")

```

Пояснення

1. **Розрахунок загальної вартості:** Для кожного запасу ми обчислюємо загальну вартість як добуток ціни за одиницю на кількість.
2. **Сортування за вартістю:** Сортуємо запаси в порядку спадання за загальною вартістю.
3. **Класифікація за ABC:** Використовуючи порогові значення, класифікуємо запаси в категорії A, B та C.

Вихід

Програма виводить кожний запас із його категорією (A, B або C) і загальною вартістю.

Отже, розглянуті моделі дуже сильно впливають на режим системності постачання матеріальних ресурсів виробничим підприємствам та організаціям. Важливо оптимізувати рівень наявних матеріальних запасів, і водночас

організувати процедуру безперебійного постачання наявних сировинних ресурсів. Вплив управління запасами на ланцюг поставок тісно пов'язаний з *постачальниками, виробниками, центрами розподілу, роздрібною торгівлею, наявністю торговельних центрів та покупців.* Усі вони мають свої специфічні вимоги до наявних сировинних ресурсів. Отже, постачальник має справу із сировиною або комплектуючими чи виробами, а також з матеріалами для технічного обслуговування, ремонту та експлуатації. Переважно ці ресурси використовуються для підтримки режиму нормального функціонування економіки. Коли постачальник відправляє товари замовнику, то зазвичай запаси перебувають певний час у дорозі. Аналогічна ситуація складається і для виробника, який виготовляє товари для технічного обслуговування, ремонту та експлуатації. Коли виробник відвантажує товар, то цей товар за звичай надходить до розподільного центру. Цей центр може реалізувати свої товари безпосередньо у режимі роздрібної торгівлі або зразу відправляти замовнику. Ресурси такого типу називаються транзитними. У розподільному центрі зазвичай матеріали, комплектуючі чи сировина не зберігаються. У такому центрі можлива процедура пакування товарів. Зазвичай центр розподілу товарів відправляє їх на ринок. У торгових підприємствах здійснюється передпродажна перевірка товарів, що надходять.

Припустимо, існує необхідність провести оцінку системи керування запасами товарів деякого торгового підприємства. Із практики відомо, що попит на товари виникає через певні інтервали часу. За наявності товару в магазині покупець має можливість здійснити покупку. Якщо його вимоги не задовольняються, то зростає негативне ставлення до цього торговельного підприємства. З часом такий негатив може наростати, що, врешті-решт, може стати причиною банкрутства підприємства. Зауважимо, що торговельне підприємство має обмежені можливості зберігати товар у складських приміщеннях. Припустимо, що максимальний рівень запасу товарів, які зберігаються на складах, становить N одиниць. Стратегія прийняття рішень про поповнення запасів полягає у періодичному перегляді їх рівня. Якщо під час оцінки ситуації із запасами товару виявилось, що кількість товарів на складі

менша за m штук, то приймається рішення про поповнення запасу товарів і, очевидно, здійснюється замовлення на додаткову доставку необхідної продукції. Процедура доставки товарів здійснюється протягом певного часу і періодично. Об'єм товарних одиниць, що замовляються, дає змогу збільшити запаси до певного рівня.

У цьому випадку завданням моделювання є встановлення такої стратегії прийняття рішень, яка дає змогу забезпечити оптимально ефективне функціонування торгового підприємства з погляду максимізації у задоволенні потреб споживачів. Саме максимізація задоволення потреб населення є цільовою функцією сформульованої стратегії.

Зауважимо, що процедура реалізації товару складається із ланцюга «замовлення товару → реалізація товару → замовлення товару». Проте товар може бути відсутній, тому спрацьовує інша конструкція «замовлення товару → відмова у продажу товару через його відсутність». Внаслідок здійснення першої послідовності підраховується кількість проданих товарів та рівень задоволення попиту на товар. Якщо ж подія «реалізація товару» не здійснюється, то здійснюється подія «відмова у продажу товару через його відсутність». Через здійснення такої негативної події підраховується рівень нереалізованого попиту на необхідну продукцію. Зрозуміло, що рівень задоволення споживацького попиту населення повинен бути значно вищий за рівень нереалізованого попиту.

Ясно, що *торговельна організація прагне до оптимізації процесу задоволення попиту споживачів, в ідеалі доводячи його до 100 %*. Отже, потрібні своєчасні управлінські вказівки в системі безперервного поповнення торгового запасу. Для цього необхідно реалізувати такі функції:

- 1) *постійний електронний моніторинг стану товарних запасів;*
- 2) *оцінка рівня достатності товарних запасів;*
- 3) *оцінка ступеня недостатності товарних запасів;*
- 4) *поповнення товарних запасів до необхідного рівня.*

На початковій стадії моделювання будемо вважати, що запас достатній, тому показник знаходиться у позиції «достатній запас». Оскільки умова «постійний електронний моніторинг стану товарних запасів» виконана, то

можливий розвиток подій такий. За позитивного виконання умови 2 (рівень запасів достатній) торговельна організація запасів не поповнює та працює у своєму нормальному режимі. Якщо ж згідно з функцією 3 виявляється, що рівень запасів недостатній, то відбувається перехід для реалізації четвертої позиції.

Рішення про достатній стан запасу товарів приймається, якщо позиція «запас» містить не менше m маркерів. У протилежному випадку приймається рішення про недостатній стан запасу і здійснюється подія «поповнення запасів до необхідного рівня». Внаслідок реалізації цієї функції здійснюється доставка товарів, і в позицію «запас» надходить відповідна кількість маркерів. Поповнення запасу відбувається до максимального рівня N штук товарів. Отже, кількість товарів, що доставляються, розраховується як максимальна кількість товарів мінус кількість товарів у запасі M , тобто $N - M$ одиниць товару. Подібний аналіз дає змогу отримати мережу Петрі, яка представлена на рис. 5.6.

5. МОДЕЛЬ ПЕТРІ

Наведена модель мережі Петрі дає реальну можливість активно керувати запасами: не накопичувати їх на складах, з одного боку, а з іншого – максимально оперативно та у повному обсязі задовольняти потреби споживачів.

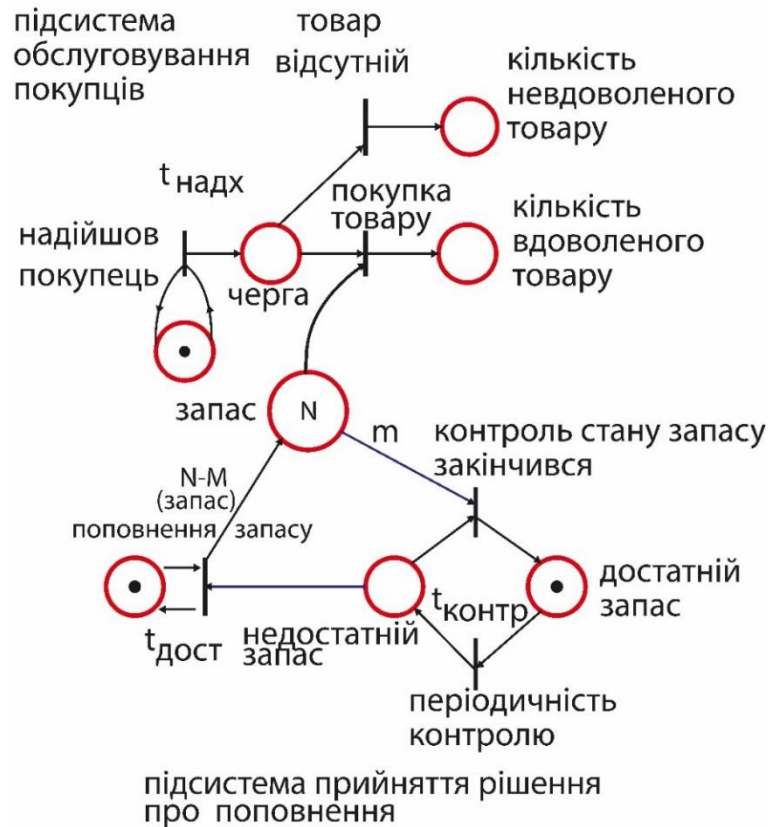


Рис. 5.6. Модель керування запасами.

Модель Петрі (або мережа Петрі) – це математична модель, яка використовується для опису й аналізу динамічних систем, зокрема тих, що мають паралельні процеси та асинхронні події. Її часто використовують для моделювання різних процесів у комп'ютерних науках, інженерії, автоматизації, а також для аналізу робочих процесів і систем, що взаємодіють у реальному часі.

Основні компоненти моделі Петрі:

1. **Місця (Places)** – позначають стан або умови у системі. На діаграмі вони позначаються колами.
2. **Переходи (Transitions)** – представляють події, що змінюють стан системи. На діаграмі вони позначаються прямокутниками або лініями.
3. **Маркування (Tokens)** – відображають поточний стан місць. Кожен токен у місці означає, що даний стан виконаний або є активним.

4. **Дуги (Arcs)** – з'єднують місця з переходами або переходи з місцями, визначаючи потік системи. Дуги можуть мати вагу, яка визначає кількість маркерів, необхідних для виконання переходу.

Принцип роботи моделі Петрі:

1. **Маркування місць** означає поточний стан системи. Токени у місцях вказують, які умови задовольняються.
2. **Активація переходів:** перехід активується, якщо у всіх місцях, з яких до нього є дуги, є достатньо маркерів.
3. **Переміщення маркерів:** якщо перехід активується, він може "спрацювати", тобто забрати маркери з вхідних місць і додати їх у вихідні місця.

6. ВИКОРИСТАННЯ ШТУЧНОГО ІНТЕЛЕКТУ У МОДЕЛЯХ КЕРУВАННЯ ЗАПАСАМИ

Керування запасами є одним із ключових елементів логістики та виробничого менеджменту. Традиційні методи управління запасами базуються на аналітичних моделях, таких як EOQ (Economic Order Quantity), ABC-аналіз чи моделі стохастичного попиту. Проте сучасні умови ринку — зростаюча невизначеність попиту, коливання цін, складність логістичних ланцюгів — вимагають більш гнучких і адаптивних рішень. У цьому контексті **штучний інтелект (ШІ)** стає потужним інструментом для оптимізації управління запасами.

Роль штучного інтелекту у системах керування запасами

III дозволяє:

- **Автоматизувати** процес прийняття рішень щодо поповнення запасів;
- **Прогнозувати попит** з високою точністю, враховуючи сезонність, тренди, акції, погодні умови, поведінку споживачів;
- **Оптимізувати витрати** на зберігання, транспортування та постачання;
- **Виявляти аномалії** у даних (наприклад, раптове зростання споживання або затримки поставок);
- **Підтримувати стратегічне планування** шляхом симуляцій і сценарного моделювання.

Методи штучного інтелекту, що застосовуються

1. Машинне навчання (Machine Learning)

- Використовується для прогнозування попиту та оптимізації розмірів замовлень.
- Алгоритми: регресія, дерева рішень, ансамблеві методи (Random Forest, XGBoost), нейронні мережі.

2. Глибоке навчання (Deep Learning)

- Моделі LSTM або GRU застосовуються для **прогнозування часових рядів попиту**.
- CNN або трансформери можуть аналізувати багатофакторні дані (ціни, рекламу, економічні індекси).

3. Нейронні мережі

- Моделюють складні залежності між факторами попиту і постачання.
- Можуть адаптуватися до змін середовища в реальному часі.

4. Системи підтримки прийняття рішень на базі ШІ

- Поєднують експертні знання та алгоритми оптимізації (наприклад, генетичні алгоритми або рої частинок) для розрахунку оптимальних обсягів замовлень.

5. Підкріплювальне навчання (Reinforcement Learning)

- Використовується для динамічного управління запасами, коли алгоритм вчиться на основі зворотного зв'язку (винагороди або штрафу за рівень запасів, витрати тощо).

Архітектура інтелектуальної системи управління запасами

Типова структура містить такі компоненти:

1. **Модуль збору даних** – збирає інформацію про продажі, постачання, ціни, погоду тощо.
2. **Модуль прогнозування** – застосовує моделі машинного навчання для оцінки попиту.
3. **Модуль оптимізації** – обчислює, коли і скільки замовити товару.
4. **Модуль симуляції** – тестує різні сценарії (наприклад, затримка постачання, зміна цін).
5. **Інтерфейс прийняття рішень** – надає користувачеві рекомендації або автоматично ініціює замовлення.

Переваги використання ШІ

- Підвищення точності прогнозів попиту на 20–50%;
- Зниження рівня запасів на складах без ризику дефіциту;
- Зменшення операційних витрат;
- Автоматизація рутинних процесів;
- Підвищення гнучкості системи у відповідь на ринкові зміни.

Приклади практичного застосування

- **Amazon** використовує алгоритми машинного навчання для прогнозування попиту та автоматичного поповнення складів.
- **Walmart** застосовує AI-системи для оптимізації ланцюгів постачання на основі аналізу реальних продажів і зовнішніх факторів.

- **Procter & Gamble** використовує нейронні мережі для керування запасами сировини в глобальних виробничих центрах.

Виклики та обмеження

- Високі вимоги до **якості даних**;
- Необхідність інтеграції ІІ з існуючими ERP-системами;
- Етичні питання автоматизації прийняття рішень;
- Потреба у фахівцях з аналітики даних та AI.

Висновки

Використання штучного інтелекту в управлінні запасами дає змогу перейти від реактивних стратегій до **прогностичних і проактивних**. Інтелектуальні моделі дозволяють не лише знижувати витрати, а й забезпечувати стійкість ланцюгів постачання в умовах невизначеності. У майбутньому інтеграція ІІ з Інтернетом речей (IoT) та блокчейном створить **повністю автономні системи управління запасами** у реальному часі.

[СММ LEC 8-5]

програма демонструє використання *Random Forest* для прогнозування попиту та порівняння двох політик керування запасами: 1. *Forecast-based* (на основі прогнозу ІІ) 2. *Baseline* (на основі середнього історичного попиту)

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt
```

```
# --- 1. Генерація синтетичних даних ---
np.random.seed(42)
n_days = 900
```

```

days = np.arange(n_days)
demand = 50 + 10*np.sin(2*np.pi*days/365) +
0.05*days + np.random.normal(0,5,n_days)
demand = np.maximum(0, demand)

# --- 2. Побудова признаков ---
df = pd.DataFrame({'demand': demand})
for lag in range(1,8):
    df[f'lag_{lag}'] = df['demand'].shift(lag)
df['rolling_mean'] =
df['demand'].shift(1).rolling(7).mean()
df['day_of_week'] = days % 7
df = df.dropna()

# --- 3. Навчання моделі RandomForest ---
train_size = int(0.8 * len(df))
X_train, X_test =
df.drop(columns='demand').iloc[:train_size],
df.drop(columns='demand').iloc[train_size:]
y_train, y_test = df['demand'].iloc[:train_size],
df['demand'].iloc[train_size:]
model = RandomForestRegressor(n_estimators=200,
random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# --- 4. Імітація керування запасами ---
def simulate_inventory(demand_series,
forecast_series, lead_time=3, holding_cost=1,
stockout_cost=5, order_cost=20):
    inventory = 100
    order_queue = []
    total_cost = 0
    inventory_history = []
    total_orders = 0

    for t in range(len(demand_series)):
        # отримання замовлень
        arrived = [q for q in order_queue if q[0]
== t]
        for _, qty in arrived:

```

```

        inventory += qty
    order_queue = [q for q in order_queue if
q[0] > t]

    # продаж
    demand_t = demand_series[t]
    sold = min(inventory, demand_t)
    inventory -= sold

    # штраф за дефіцит
    stockout = max(0, demand_t - sold)
    total_cost += stockout * stockout_cost

    # утримання запасів
    total_cost += inventory * holding_cost

    # політика поповнення
    if t < len(forecast_series) - lead_time:
        R =
forecast_series[t:t+lead_time].sum() + 10
    else:
        R = 50
    if inventory < R:
        Q = R - inventory + 50
        order_queue.append((t + lead_time, Q))
        total_cost += order_cost
        total_orders += 1

    inventory_history.append(inventory)

    return total_cost, total_orders,
inventory_history

actual_demand = y_test.values
forecast_demand = y_pred

cost_forecast, orders_forecast, inv_forecast =
simulate_inventory(actual_demand, forecast_demand)
cost_baseline, orders_baseline, inv_baseline =
simulate_inventory(actual_demand,
np.full_like(forecast_demand, np.mean(y_train)))

```

```
# --- 5. Вивід результатів ---
print("MAE:", np.mean(np.abs(y_test - y_pred)))
print(f"Forecast-based policy: total cost =
{cost_forecast:.2f}, total orders =
{orders_forecast}")
print(f"Baseline policy: total cost =
{cost_baseline:.2f}, total orders =
{orders_baseline}")

# --- 6. Візуалізація ---
plt.figure(figsize=(10,5))
plt.plot(y_test.values[:200], label="Actual
demand")
plt.plot(y_pred[:200], label="Forecast")
plt.legend()
plt.title("Прогноз попиту vs фактичні дані")
plt.xlabel("Дні (тестовий період)")
plt.ylabel("Попит")
plt.show()

plt.figure(figsize=(10,5))
plt.plot(inv_forecast[:200], label="Forecast-based
inventory")
plt.plot(inv_baseline[:200], label="Baseline
inventory")
plt.legend()
plt.title("Рівень запасів у двох політиках")
plt.xlabel("Дні")
plt.ylabel("Запаси")
plt.show()
```

